

The logo for Fintrack, consisting of the word "Fintrack" in white text on a green rectangular background.

Fintrack

Explicacao do Projeto Fintrack

Aplicativo Android em Java para controle financeiro mensal, com Firebase Authentication e Cloud Firestore.

Documento gerado para resumir a arquitetura, telas, modelos de dados e principais fluxos implementados.

1. Visao Geral

O Fintrack e um aplicativo Android desenvolvido em Java para registrar salario mensal, gastos por categoria, formas de pagamento e saldo disponivel. O app usa Firebase Authentication para login/cadastro e Cloud Firestore para persistir os dados do usuario.

- Cada usuario possui seus proprios documentos dentro de usuarios/{uid}.
- Os gastos e salarios sao separados por mes e ano usando o campo mesAno.
- A tela principal atualiza automaticamente os totais a partir dos listeners do Firestore.
- Meses anteriores continuam salvos para consulta historica futura.

2. Principais Telas

Tela	Arquivo	Responsabilidade
Splash	SplashActivity / activity_splash.xml	Exibe a logo e o nome do app antes do login.
Login	MainActivity / activity_main.xml	Autentica o usuario com Firebase Authentication.
Cadastro	CadastroActivity / activity_cadastrar.xml	Cria conta e salva dados basicos do usuario.
Principal	PrincipalActivity / activity_principal.xml	Mostra resumo mensal, categorias, saldo disponivel e atalhos.
Adicionar gasto	AdicionarGastoActivity / activity_adicionar_gasto.xml	Registra gasto com categoria, pagamento e parcelas.
Adicionar salario	AdicionarSalarioActivity / activity_adicionar_salario.xml	Salva salario do mes atual.
Gastos por categoria	GastosCategoriaActivity / activity_gastos_categoria.xml	Mostra gastos de uma categoria e permite deletar.

3. Fluxo de Autenticacao

O usuario inicia na SplashActivity e depois e direcionado para a MainActivity. A tela de login usa FirebaseAuth.signInWithEmailAndPassword. Quando o login e bem-sucedido, o app abre a PrincipalActivity.

No cadastro, CadastroActivity cria o usuario com FirebaseAuth.createUserWithEmailAndPassword e salva um documento Usuario em usuarios/{uid}.

4. Organizacao no Firestore

Colecao / Documento	Conteudo
usuarios/{uid}	Dados basicos do usuario, como nome e email.
usuarios/{uid}/gastos/{id}	Cada gasto individual, com valor, categoria, data, forma de pagamento, parcelas e mesAno.
usuarios/{uid}/salarios/{mesAno}	Salario registrado para o mes correspondente.
usuarios/{uid}/resumosMensais/{mesAno}	Resumo calculado: salario, total de gastos e saldo restante.

5. Models

Classe	Campos principais	Uso
Usuario	nome, email	Representa o documento principal do usuario.
Gasto	descricao, valor, categoria, formaPagamento, parcelas, data, mes, ano, mesAno	Representa um gasto salvo no Firestore.
Salario	valor, data, mes, ano, mesAno	Representa o salario mensal do usuario.
ResumoMensal	salario, totalGastos, saldoRestante, mes, ano, mesAno, atualizadoEm	Representa o historico mensal calculado.

6. Fluxo de Gastos

Ao adicionar um gasto, o usuario informa descricao, valor, categoria, forma de pagamento e, se a forma for Credito, a quantidade de parcelas. O valor usa mascara automatica em reais, convertendo a digitacao para formato monetario.

- O gasto e salvo em usuarios/{uid}/gastos.
- O campo mesAno permite separar gastos por mes sem apagar registros antigos.
- A tela principal escuta alteracoes e recalcula os totais automaticamente.
- Na listagem por categoria, o usuario pode tocar em um gasto e deletar o documento.

7. Salario, Saldo e Historico Mensal

O salario do mes e salvo no documento usuarios/{uid}/salarios/{mesAno}. A PrincipalActivity calcula o saldo disponivel subtraindo os gastos do mes atual do salario registrado.

- Se nao houver salario no mes atual, o salario considerado e zero.
- Se nao houver gastos no mes atual, os blocos de gastos ficam ocultos e o saldo usa apenas o salario.
- Se os gastos ultrapassarem o salario, o saldo fica negativo e aparece em vermelho.
- O resumo mensal e salvo em resumosMensais/{mesAno}, preservando historico financeiro.

8. Atualizacao Automatica

A tela principal usa addSnapshotListener para observar salarios e gastos. Isso significa que qualquer criacao, alteracao ou exclusao no Firestore dispara a atualizacao da interface em tempo real.

- Ao adicionar gasto: total de gastos, categorias e saldo sao recalculados.
- Ao deletar gasto: o documento some da lista e os totais diminuem automaticamente.
- Ao adicionar salario: o saldo disponivel e atualizado sem precisar reiniciar o app.

9. Padrao Visual

As telas seguem um padrao visual consistente: fundo claro, cards brancos, botoes verdes com a cor da marca, tipografia simples e icones vetoriais para categorias e a logo.

- Cor principal: #10B981.
- Logo: carteira verde em fundo branco.

- Botoes principais: backgroundTint @color/emerald_green.

10. Pontos para Evolucao

- Criar tela de historico para consultar meses anteriores.
- Permitir editar gastos e salario ja cadastrados.
- Adicionar filtro por forma de pagamento.
- Gerar relatorios mensais em PDF diretamente pelo app.
- Adicionar graficos para visualizar categorias ao longo do tempo.